

TER: Détection de communauté coloré

Valentin Hesters 20201346
valentin.hesters@gmail.com

Sara Firoud 20235275
firoud.saraa@gmail.com
Encadrant: Franck Delaplace

15 Mai 2024

Sommaire

1	Introduction	1
1.1	Objectif du Travail d'étude et de recherche	2
1.2	Résumé des travaux de départ	2
1.2.1	Formalisation	2
1.2.2	Calcul des probabilités de coloration aléatoire	4
1.2.3	CHROCODE: Algorithme de détection des communautés colorées	5
2	L'algorithme génétique	7
2.1	Introduction	7
2.2	Méthodologie	8
2.3	Population Initiale	9
2.3.1	Initialisation	9
2.3.2	Evaluation de la validité d'un génome	9
2.3.3	Génération d'un génome	10
2.4	Opérateurs de l'algorithme génétique	13
2.4.1	Sélection par tournoi	13
2.4.2	crossover	14
2.4.3	Mutation	16
3	Résultats	17
4	Conclusion	21

1 Introduction

Vous retrouverez le code à cette adresse: <https://github.com/suprawall/AGCodeTER>

1.1 Objectif du Travail d'étude et de recherche

Notre projet porte sur la détection des structures communautaires colorées dans les réseaux complexes. Afin de bien introduire et de présenter notre approche et les avancées de nos recherches, il est essentiel de définir certaines notions fondamentales.

D'abord, les réseaux complexes sont souvent représentés comme des graphes qui modélisent des interactions entre agents. Ils servent dans de nombreux domaines d'études telles que la sociologie (relations sociales), l'informatique (internet, réseaux sociaux...) et la biologie (réseaux neuronaux ou métaboliques). Au sein de ces réseaux, les nœuds sont souvent organisés en communautés étroitement liées, lesquels reflètent des fonctions ou des objectifs communs, révélant la structure organisationnelle du réseau. Identifier une partition des nœuds nous permet donc de mieux comprendre cette structure.

Formellement, notre objectif est d'identifier des structures communautaires dans un réseau. Ces structures sont définies comme une partition des sommets d'un graphe. Une communauté se caractérise généralement par une densité de connexions plus élevée entre ses nœuds qu'avec ceux situés à sa périphérie. Cette densité interne doit surpasser la densité de connexion entre les communautés. Les algorithmes de détection de communauté exploitent cette disparité de densité pour identifier les communautés au sein d'un réseau.

Monsieur Delaplace, a développé un algorithme nommé Chrocode, qui aborde le problème de la formation des communautés en intégrant les couleurs comme propriétés partagées. Cet algorithme vise à maximiser le regroupement de nœuds de même couleur au sein des communautés, en utilisant une mesure appelée entropie chromatique pour évaluer la qualité de la structure communautaire en fonction de la contrainte de couleur.

Pour nous, ce code constitue un guide, offrant une référence sur la manière de trouver une partition de communautés efficace dans notre graphe, basée sur l'entropie chromatique.

Notre objectif est de comprendre en profondeur l'algorithme Chrocode et de nous en inspirer pour développer une méthode alternative capable pour construire des structures communautaires de manière tout aussi efficace, voire plus. Pour ce faire, nous avons exploré diverses pistes et méthodes suggérées par notre professeur, ainsi que celles découvertes au cours de nos recherches.

Dans ce rapport, nous allons tout d'abord commencer par expliquer les notions importantes introduites pour l'algorithme Chrocode. Ensuite, nous détaillerons l'approche que nous avons choisi de suivre, les résultats obtenus et nos conclusions sur le sujet.

1.2 Résumé des travaux de départ

1.2.1 Formalisation

Soit $G = \langle V, E \rangle$ un graphe où V est l'ensemble des sommets et E l'ensemble des arêtes. On introduit la notion de communauté notée p qui est un sous-ensemble de V . P est l'ensemble de ces communautés. On a alors ces propriétés:

$$\bigcup_{p_i \in P} p_i = V \quad \text{et} \quad \forall p_i, p_j \in P : (p_i \cap p_j \neq \emptyset) \implies (p_i = p_j)$$

C'est à dire que tous les sommets du graphe se trouvent dans une communauté. De plus deux communautés différentes n'ont aucun sommets en communs.

L'ensemble des couleurs C est représentés par un intervalle $[1, r]$ où chaque couleur est associée à un entier. Ainsi la coloration d'une communauté notée c_p est définie par:

$$c_p = \{v \mapsto c(v) \mid v \in p\}$$

où $c(v)$ est la couleur associée au sommet v .

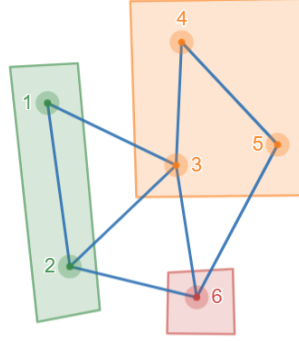


Figure 1: *Exemple d'un graphe coloré*

Le graphe de la Figure 1 est composé de trois communautés:
 $P = \{p_1 = \{1, 2\}, p_2 = \{3, 4, 5\}, p_3 = \{6\}\}$, et d'un profil de couleur:
 $c = \{1 \rightarrow 1, 2 \rightarrow 1, 3 \rightarrow 2, 4 \rightarrow 2, 5 \rightarrow 2, 6 \rightarrow 3\}$, que l'on peut représenter sous forme de vecteur: $(1, 1, 2, 2, 2, 3)$.

Fonction Chromatique : $\chi_c = \bigcup_{k \in C} \mathbf{Count}(c, k)$

elle compte le nombre d'occurrence de chaque couleur k dans un profil c . Dans cet exemple on a : $\chi_c = \{1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1\}$

De plus on introduit les notions de *d-coloring community*: $\exists k \in C : \chi_{c_p}(k) = d$ et de *d-dominant*: $\exists k \in C : \chi_c(k) = d \wedge \forall k' \in C : \chi_c(k') \leq d$.

Ainsi, on peut dire du graphe de la Figure 1 qu'il est un *2-coloring profile* pour la couleur 1, un *3-coloring profile* pour la couleur 2 et un *1-coloring profile* pour la couleur 3. La couleur 2 est dominante et donc ce graphe est *3-dominant*.

Entropie Chromatique : La signification d'une structure communautaire

chromatique sera déduite par une mesure d'entropie. Elle se rapportera à une quantité de chaos et d'aléatoire et sera donc liée à la coloration d'une communauté obtenue par chance. Ainsi, on cherchera à minimiser cette valeur dans l'algorithme pour évaluer l'intentionnalité derrière la répartition des couleurs au sein d'une communauté.

On fixe la mesure d'intentionnalité à $I(p) = \log_2(1 - p)$, la mesure d'entropie chromatique sera : $H(p) = -pI(p) = -p \log_2(1 - p)$.

1.2.2 Calcul des probabilités de coloration aléatoire

On définit la probabilité de générer une *d-colorful* communauté avec une couleur r au hasard par le nombre de cas favorable divisé par le nombre de cas possible. Le nombre de cas possible correspond au cardinal de l'énumération complète des combinaisons possibles de coloration des sommets parmi r couleurs, c'est à dire r^n .

Pour définir le nombre de cas favorable on s'intéresse à deux énumérations possibles: celle des communautés *d-colorful* et celle des communautés *d-dominant* de taille n sur r couleurs.

Enumération des *d-colorful* communautés:

Soit D_k l'ensemble des communautés avec d sommets de couleur k . l'union de ces sets: $|\bigcup_{k=1}^r D_k|$ nous permet d'avoir le décompte de toutes les communautés qui contiennent un *d-color profile*. Cependant le fait qu'elles peuvent avoir un plusieurs *d-profile* pour différentes couleurs induit une intersection des ensembles D_k . En se basant sur la *Poincaré sieve* on obtient cette formule d'union:

$$\left| \bigcup_{k=1}^r D_k \right| = \sum_{k=1}^r (-1)^k \sum_{1 \leq i_1 \leq \dots \leq i_j \leq \dots \leq i_k \leq r} |D_{i_1} \cap \dots \cap D_{i_j} \cap \dots \cap D_{i_k}| \quad (1)$$

Grâce aux formules combinatoires des ensembles et de l'intersection de ses ensembles on peut en déduire le théorème suivant:

le décompte des d-colorful communautés de taille n avec r couleurs est donné par la fonction κ :

$$\kappa(r, n, d) = \sum_{k=1}^{\min(r, \lfloor \frac{n}{d} \rfloor)} \frac{(-1)^{k-1} \binom{r}{k} n! (r-k)^{n-kd}}{(n-kd)! (d!)^k} \quad (2)$$

Enumération des *d-dominants* communautés:

la fonction γ utilise la notion de signature σ : En effet si deux fonction chromatique de deux communautés p et q entretiennent cette propriété:

$\chi_{cp} = \pi \circ \chi_c$ avec π une permutation de couleur sur $\pi : C \rightarrow C$ alors elles partagent les mêmes propriétés de dominations.

La signature chromatique a pour objectif de repérer cette équivalence.

$$\sigma_p = \mathbf{Sort} \circ \mathbf{Img}\chi_c \quad (3)$$

Elle vérifie ces propriétés:

$$\begin{cases} \sigma(r) = d \wedge \\ \prod_{i=1}^r \sigma(i) = n \wedge \\ \forall 1 \leq i \leq r : \sigma(i) \leq d \wedge \\ \forall 1 \leq i, j \leq r : i \leq j \Rightarrow \sigma(i) \leq \sigma(j). \end{cases}$$

Ainsi, le graphe de la Figure 1 a pour signature chromatique : (1,2,3). Plusieurs fonctions peuvent avoir la même signature, par exemple

$\chi_c = \{1 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 2\}$ donne aussi: (1,2,3). De plus nous avons besoin de pouvoir compter le nombre de signature possible, pour cela nous utiliserons la fonction $S_{r,n,d}$ qui représentera l'ensemble des signatures possibles selon les paramètres r, n et d . Elle sera calculée par un algorithme.

Par exemple $S_{r,n,d}$ avec $r = 4, n = 7, d = 3$ donne : $S_{4,7,3} = \{(0, 1, 3, 3), (1, 1, 2, 3), (0, 2, 2, 3)\}$.

Le décompte des d -dominants communautés se fait en deux étapes: d'abord à partir des signatures nous comptons les fonctions chromatiques qui leurs correspondent, ensuite nous comptons tous les profils de couleurs qui peuvent amener à ces fonctions chromatiques. cela nous donne la formule suivante:

le décompte de tous les d -dominant communautés de taille n avec r couleurs est donné par la fonction γ :

$$\gamma(r, n, d) = n!r! \sum_{\sigma \in S_{r,n,d}} \frac{1}{\prod_{s \in \mathbf{Img} \circ \mathbf{Count}(\sigma)} s! \prod_{i=1}^r \sigma(i)!} \quad (4)$$

1.2.3 CHROCODE: Algorithme de détection des communautés colorées

L'algorithme de détection des communautés chromatiques (CHRODE) a pour objectif de partitionner un graphe coloré tout en minimisant l'entropie chromatique H .

Ce processus se déroule en deux phases distinctes :

Phase 1 : Structure communautaire monochrome connectée : À partir d'un graphique coloré $\langle V, E, C \rangle$ des partitions initiales sont générées en regroupant les nœuds connectés et partageant la même couleur. Et comme résultat de cette étape-là, des communautés monochromes sont obtenues.

Phase 2 : Fusion des communautés monochromes : on définit un graphe de

quotient Q où chaque communauté monochromes P précédemment obtenu devient un nœud de ce graphe ($Q = \langle P, EP \rangle$). Ces communautés sont ensuite fusionnées itérativement pour réduire l'entropie chromatique, en tenant compte des liens existants entre elles. Un lien est établi entre deux nœuds communautaires s'il existe déjà un lien entre certains nœuds composant les communautés respectives $EP = \{(p_i, p_j) \mid \exists (v_i, v_j) \in E : v_i \in p_i \wedge v_j \in p_j\}$

À chaque itération, la communauté P avec l'entropie chromatique H la plus élevée est sélectionnée et fusionnée avec son voisin, minimisant ainsi au maximum son entropie. Ces deux communautés forment alors un nouveau nœud communautaire correspondant à leur union. Ce processus se répète jusqu'à atteindre un point de stabilité aucune fusion supplémentaire ne puisse réduire davantage l'entropie, aboutissant à la formation de graphes de quotient avec des communautés monochromes.

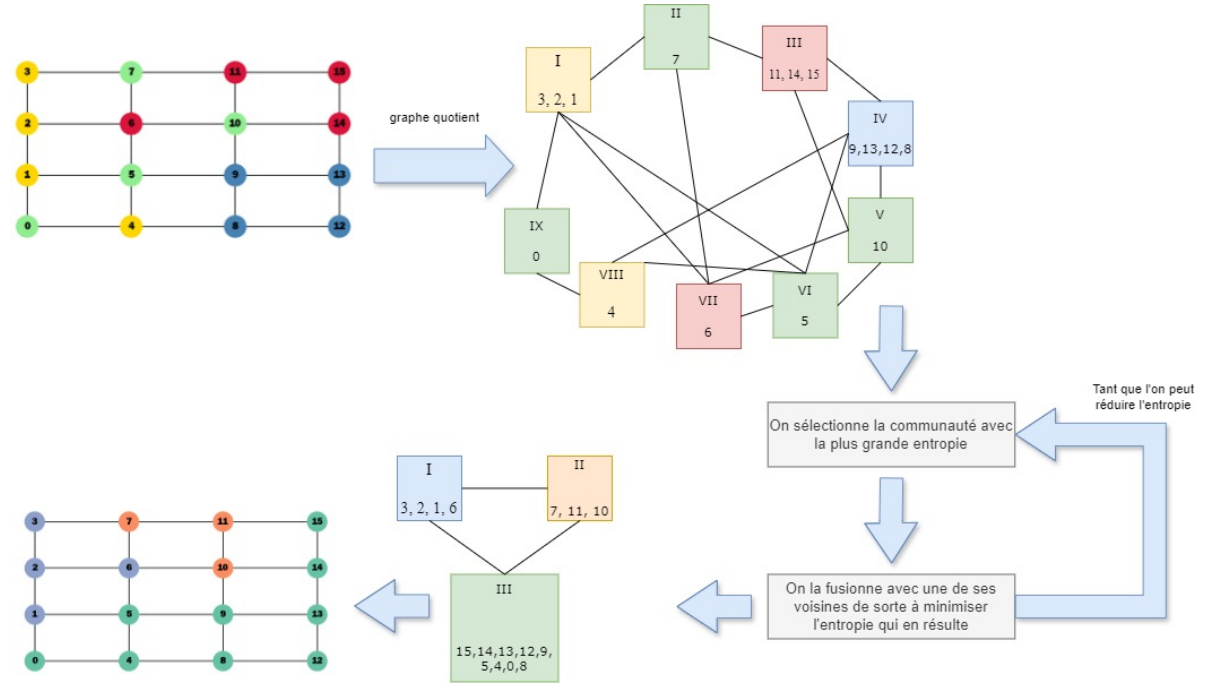


Figure 2: *Algorithme chrocode.*

A partir de ce graphe coloré $G = \langle V, E, C \rangle$ on construit le graphe de quotient à 9 états qui est illustré dans la figure 2. Rappelons qu'un état de ce graphe n'est connecté qu'avec des états de couleurs différentes. Tout d'abord, la constitution du graphe quotient, représente la première phase de l'algorithme où la complexité est égale à $O(|E|)$ puisque l'algorithme par-

court l'ensemble des nœuds. Supposons maintenant le pire cas, c'est-à-dire que le graphe quotient n'est constitué que de communautés monochromes d'un seul nœud formant un ensemble de singletons. Ce cas se produit lorsque chaque nœud du graphe de départ a des voisins de couleurs différentes par rapport à lui. Si on suppose également qu'à chaque étape seule la fusion de deux communautés se produit, la complexité dans ce cas-là est : $O(|V|^2(|E| + |V| \log(|V|)))$.

L'algorithme CHROCODE procède à la fusion des communautés en sélectionnant de manière itérative celle avec la plus grande valeur d'entropie et sa voisine qui réduit au maximum cet entropie. Le processus se termine enfin lorsqu'aucune fusion supplémentaire ne peut diminuer l'entropie.

2 L'algorithme génétique

2.1 Introduction

Le but de notre travail est de trouver et d'implémenter une méthode qui améliorerait la distribution en communauté de CHROCODE. Nous avons d'abord pensé à implémenter une nouvelle fonction de fitness multi-objectif qui combinerait plusieurs variables : l'entropie, la modularité, la densité et la conductance. Ces mesures sont généralement utilisées dans l'analyse des systèmes complexes, mais s'avèrent peu efficaces quand il s'agit de distribuer les nœuds en communautés. Nous nous sommes alors tourné vers la 2ème solution qui est d'optimiser le problème via une méta-heuristique.

Les algorithmes génétiques s'inspirent des principes de l'évolution et de la sélection naturelle introduite par Darwin pour résoudre des problèmes complexes d'optimisation. Ils manipulent des générations d'individus codant chacun une solution valide du problème. Ces individus, appelés génomes, subissent des processus de sélection, de mutation et de croisement qui vont permettre d'améliorer au fur et à mesure du temps la performance globale d'une génération et de s'approcher le plus possible d'une solution optimale au problème. Chaque individu sera mesuré ici par une fonction de fitness basée sur l'entropie, similaire à celle utilisée dans l'algorithme CHROCODE, qu'il devra minimiser pour survivre et se reproduire.

Il convient ici de choisir la manière la plus simple de représenter chaque génome pour coder une solution du problème. Ainsi, le numéro du gène représentera le nœud correspondant dans le graphe, et la valeur de ce gène sera la communauté à laquelle le nœud appartient.



Figure 3: *exemple d'un génome solution*

2.2 Méthodologie

Nous avons commencer par implémenter une version de base de l'algorithme génétique, avec des opérateurs non optimisé, afin de se familiariser avec cette méta-heuristique. Cela nous a permis d'éclaircir les points importants à travailler pour réussir cette implémentation. L'objectif principal est de trouver l'individu parmi les générations successives qui minimise le plus l'entropie et qui doit donc garantir une bonne répartition en communauté du graphe de départ. Notre implémentation initiale échoué sur deux points importants:

- La minimisation de l'entropie: la distribution en communauté n'était pas satisfaisante.
- la viabilité des solutions: en effet la répartition se doit de suivre une règle de connectivité des communautés qui stipule que chaque noeuds doit être voisin avec au moins un autre noeud sa communauté.

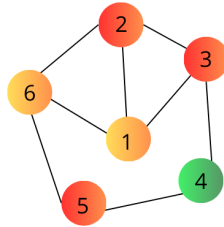


Figure 4: *exemple de viabilité*

Ainsi, la solution ci-dessus est fausse, car le nœud 5 appartient à la même communauté que les nœuds 1 et 2 bien qu'ils ne sont pas voisins.

La satisfaction sur ses deux points a occupé la majeure partis de notre travail sur le TER. La stratégie que nous avons mise en place est de générer une population initiale valide pour ensuite utiliser des opérateurs de sélections de

croisements et de mutations optimisés qui conserveront cette contrainte de connectivité tout en cherchant à regrouper les individus afin de minimiser l'entropie.

2.3 Population Initiale

2.3.1 Initialisation

La première étape est donc de générer une population initiale aléatoire où chaque génome code une solution valide sur le graphe de départ. Pour pouvoir initialiser un génome nous avons besoin de savoir dans quel intervalle nous devons choisir une communauté à attribuer à chacun de ses gènes, c'est-à-dire qu'elle est le nombre maximal de communauté que l'on considère dans le graphe. Pour ce faire, on propose de limiter cet intervalle au nombre d'états du graphe quotient. En effet, la coloration du graphe de départ se fait aléatoirement; il est donc normal de supposer qu'une solution qui propose plus de communauté qu'il n'y en a aléatoirement au départ va en contradiction avec le but de minimisation de l'entropie, qui est une valeur qui mesure justement à quel point une distribution de communauté a été générée au hasard.

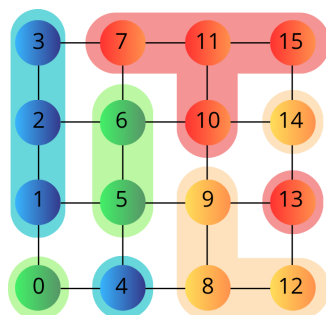


Figure 5: *Nombre maximum de communauté*

Ainsi, avec cette coloration de départ sur ce graphe, on sait que chaque gène se verra attribuer une communauté entre 1 et 8.

2.3.2 Evaluation de la validité d'un génome

Pour pouvoir générer la population initiale, on a besoin d'avoir un moyen d'évaluer la validité d'un génome. On suit les étapes suivantes : On commence par construire un ensemble de communautés à partir du génome à vérifier de la même manière que lorsqu'on construit un graphe quotient. Ensuite, il suffit de vérifier que les communautés trouvées sont toutes différentes les unes

des autres. Dans le cas contraire, cela indique que nous avons au moins une discontinuité dans une communauté.

Algorithm 1: EstValide

Input: Génome[] : Tableau représentant le génome
 Graphe du problème
Output: Viablié du génome

```

1 Ensemble[ ]
2 Génome[ ]
3 // Remplissage dans "Ensemble" des communautés jointes
4 for i ← 0 to longueur du génome do
5   if genome[i] est dans Ensemble then
6     | passer à l'itération suivante
7   Frontiere[ ]
8   Explorer[ ]
9   Frontiere ← genome[i]
10  while Frontiere n'est pas vide do
11    n ← Defiler(Frontiere)
12    Explorer ← n
13    Voisins ← Les voisins de n dans le graphe
14    for tt les voisins "v" dans Voisins do
15      if genome[v] == genome[n] and "v" n'est pas dans Explorer
16        then
17          | Frontiere ← v
18    Ensemble ← Frontiere
19  // Vérifications de la connectivité des communautés de
20  // "Ensemble"
21  for e1 dans Ensemble do
22    for e2 dans Ensemble do
23      if e1 != e2 and e1 et e2 contiennent des noeuds de la même
24      communauté then
25        | return False
26  return True

```

2.3.3 Génération d'un génome

Nous savons qu'un algorithme génétique doit se composer d'opérateurs stochastiques pour pouvoir fonctionner correctement, car il consiste à reproduire des comportements naturels. Pour générer un génome la méthode qui semble le plus adaptée à cette contrainte est celle qui consiste à générer, sur une taille de population n , des génomes aléatoirement jusqu'à en obtenir n valides. Cependant, elle est évidemment impossible en temps raisonnable.

Il est également impossible de choisir aléatoirement tous les nœuds. En effet, le nombre de communautés dans le graphe quotient est nécessairement plus petit que le nombre de nœuds total du graphe, ainsi lorsque le nombre total de communautés possible a été attribué, il restera encore des nœuds sans communauté ; donc si le nœud choisi aléatoirement n'est voisin qu'avec des nœuds qui n'ont pas encore reçu de communauté, alors ce nœud ne pourra pas se voir affecter une communauté qui rend le génome viable.

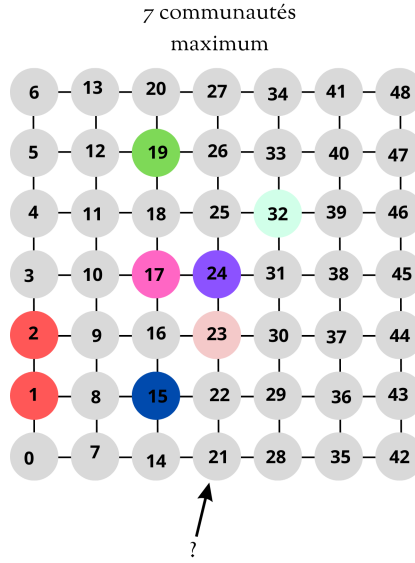


Figure 6: *Erreur de génération*

Ici, le nombre de communautés maximum est égal à 7, le nœud choisi aléatoirement pour être affecté est le 21. Toutes les communautés possibles ont déjà été attribuées. Comme le nœud 21 n'est voisin avec aucun autre nœud qui a reçu une attribution, il ne pourra jamais rendre le génome viable. On en conclut que pour la génération d'un génome la stochasticité voulu par l'algorithme génétique ne peut se faire que sur l'attribution des couleurs et non sur la sélection du nœud à attribuer.

Notre approche a donc été de parcourir le graphe de voisins en voisins en choisissant pour chaque nœud une communauté au hasard jusqu'à en trouver une qui garde la viabilité du génome. Le parcours du graphe est un parcours en Largeur et on choisit de commencer par un nœud aléatoire.

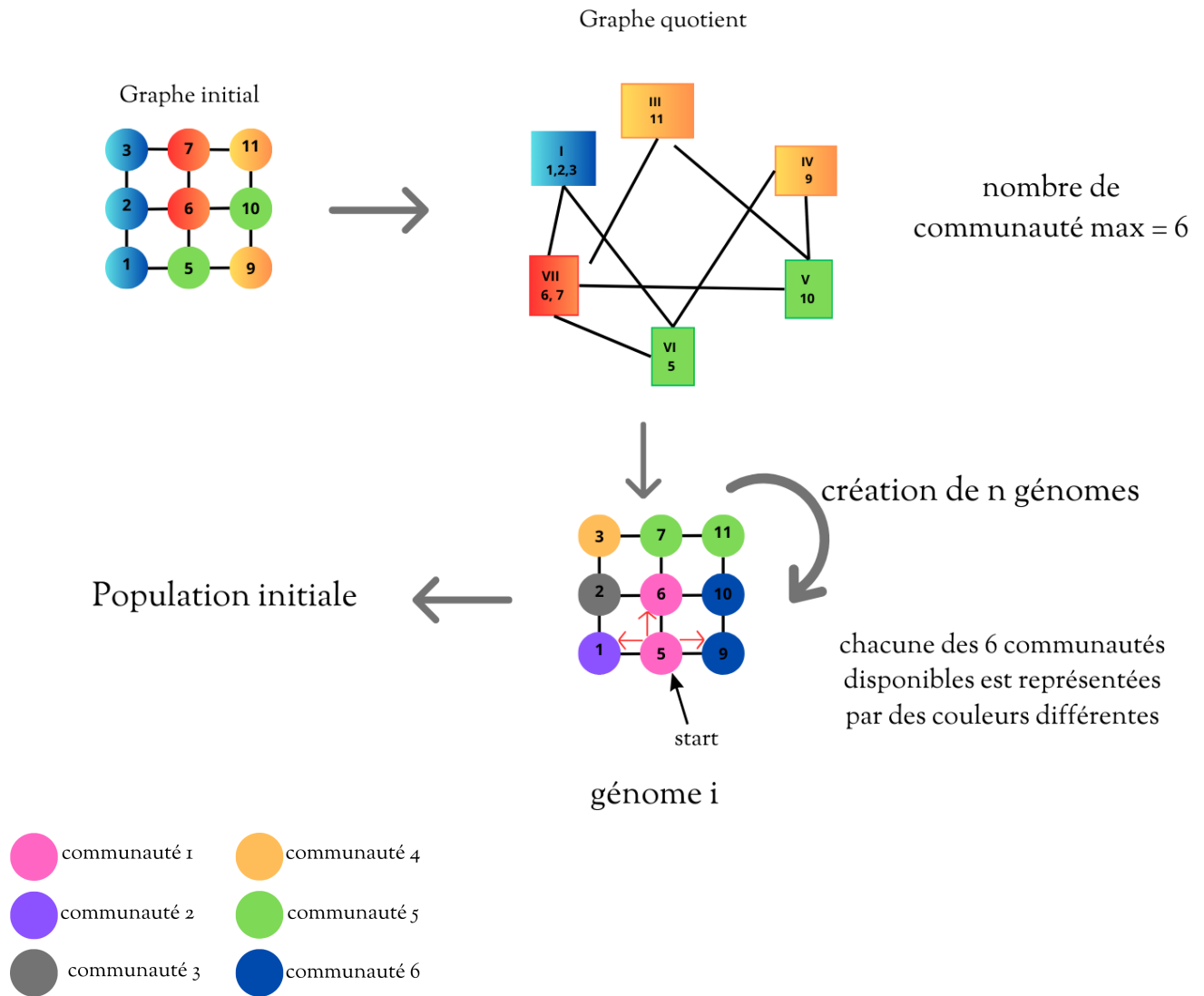


Figure 7: *initialisation*

Dans la *Figure 7* on choisit de commencer par le nœud 5 qui pourra se voir attribuer n'importe quelle communauté, ici la 1. Ensuite, on attribue une communauté à ses voisins, ainsi si le nœud 1 prend la communauté 2, les nœuds suivants (6 et 9) ne pourront pas la prendre, car ils ne sont pas voisins et le nœud 5 attribué à rose ne pourra pas faire le lien. L'inconvénient de cette méthode est que le nœud 1 et 6 par exemple aurait potentiellement pu appartenir à la même communauté si le nœud 2 avait servi de lien. Cependant, on parcourt le graphe de voisins en voisins et donc l'attribution de plusieurs nœuds à une

même communauté ne peut se faire également que de voisins en voisins.
Un nombre élevé d'individus dans la population initiale permet de remédier à ce problème en augmentant le nombre de combinaisons possible.

2.4 Opérateurs de l'algorithme génétique

2.4.1 Sélection par tournoi

Une fois la population initiale générée, il convient de lancer l'itération de l'algorithme génétique. La première étape est la sélection des individus survivants à la prochaine génération et qui vont être choisis pour se reproduire. Pour ce faire, on utilise l'entropie utilisée dans CHROCODE comme fonction de fitness, qui servira à évaluer la performance d'un génome individu. On rappelle que l'entropie se définit par : $H(p) = -pI(p) = -p \log_2(1-p)$. On remarque que si un génome code une solution où au moins une communauté n'est composée que d'un nœud, elle aura une entropie infinie, car la probabilité qu'elle soit coloriée de cette manière est de 1.

Il n'est pas intéressant de choisir les n meilleurs génomes de la population en cours pour ensuite les faire se reproduire entre eux. Il est toujours important de garder de la stochasticité et cela passe par des opérateurs adaptés. Nous avons choisi d'utiliser la méthode de sélection par tournoi. C'est une méthode simple de sélection qui choisit parmi k individus pris aléatoirement dans la population en cours, et qui va placer le meilleur d'entre eux dans la génération suivante. Dans les premières itérations de l'algorithme, il est tout à fait possible que chacun des individus participants au tournoi ait une entropie infinie, dans ce cas, on en retourne un au hasard.

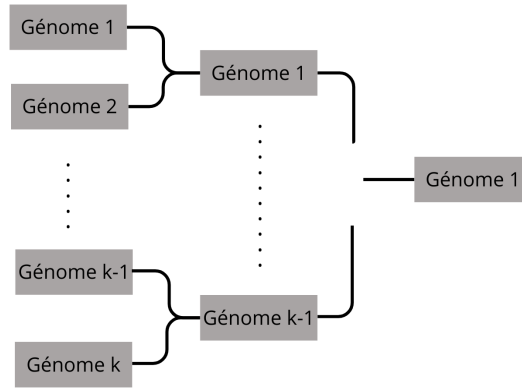


Figure 8: *Tournoi*

Le nombre d'individus qui survivent à la génération suivante est habituellement défini par un taux darwinien dans les algorithmes génétiques. Nous

avons fixé ce taux à 1 pour avoir la même taille de population pour toutes les générations successives. De plus, les individus de l'ancienne génération sur laquelle on applique la sélection ne sont pas remplacés et peuvent donc être choisis plusieurs fois et apparaître fréquemment dans la nouvelle génération. On remarque bien que le critère de réussite ici est le paramètre k , le nombre d'individus participants à chaque tournoi. Si ce nombre est élevé, les individus les plus mauvais n'auront presque aucune de chance de survivre, car ils seront automatiquement dans un tournoi avec d'autres individus meilleurs qu'eux. Cette solution n'est pas la meilleure, car tend à favoriser largement les génomes avec la meilleure entropie qui sont ceux qui ont de plus grandes communautés et donc un nombre faible de communautés différentes : on se retrouve avec une population homogène sans diversité génétique et l'algorithme convergera vite vers un optimum local. Au contraire choisir un k faible favorisera la diversité génétique au prix d'une convergence vers un optimum plus longue. Dans notre cas, nous avons pu laisser le nombre de participants à 2 car la tendance à regrouper les communautés entre elles et à minimiser leurs nombres ce fait via les deux opérateurs suivants que sont le crossover et la mutation.

2.4.2 crossover

Le processus de crossing-over reproduit le phénomène de reproduction. Il s'opère sur deux individus de la génération survivante à la sélection et retourne leurs deux enfants. Le but ici est de créer de nouveaux individus robustes qui seront plus aptes à survivre aux générations suivantes. Ainsi, nous cherchons à minimiser le nombre de communautés ce qui tend à minimiser l'entropie et donc à améliorer la solution. La complexité de la mise en place de cet opérateur est d'assurer la viabilité des deux génomes qui en découleront. En effet, le croisement doit se faire entre deux individus. Bien qu'ils sont assurés de représenter eux même une solution valide, leurs croisements contient des risques. La difficulté est de trouver un opérateur simple, qui garde de la stochasticité et qui retourne un enfant valide.

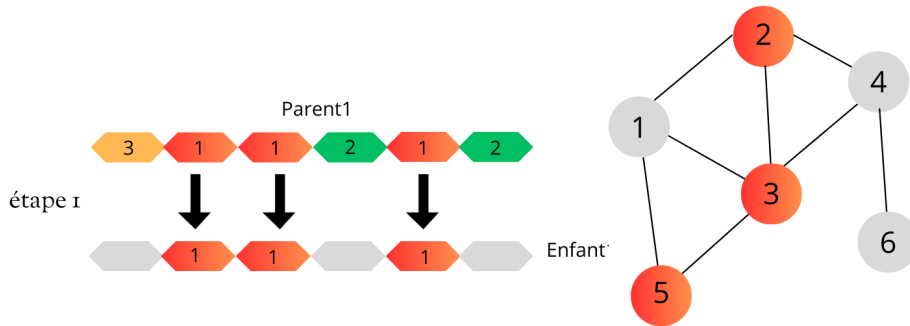


Figure 9: *Crossover étape 1*

On effectue le croisement en plusieurs étapes, la première est décrite dans la *Figure 9*, et consiste à choisir une communauté du parent et de recopier tous les nœuds appartenants à cette communauté dans l'enfant. On doit retourner deux enfants donc cette étape sera faite par le parent 1 pour l'enfant 1 et par le parent 2 pour l'enfant 2. Plusieurs manières de choisir la communauté à conserver sont possibles, par exemple compter le nombre de nœuds présent dans chaque communauté ou calculer l'entropie de chaque communauté... Notre première approche a été la plus simple et nous avons choisis de prioriser la plus grande communauté, car nous cherchons toujours à faire converger le nombre de communautés total d'un génome et la plus grande n'est pas forcément celle qui a la meilleure entropie. Pour enlever du déterminisme à cet opérateur nous donnons un poids à chaque communauté présente dans le génome en fonction de nombre de nœuds qu'elle contient, ici :

- communauté 1: 3 noeuds
- communauté 2: 2 noeuds
- communauté 3: 1 noeud

On la sélectionne via une méthode de roue biaisé. La plus grande communauté se verra attribuée un poids de 5 tandis que tous les autres auront un poids de 1. Dans les premières générations, les génomes contiennent un grand nombre de communautés et la plus prépondérante n'est pas choisie dans la grande majorité des cas, ainsi notre population se diversifie beaucoup avant de converger vers un optimum.

La deuxième étape doit faire appel au deuxième parent :

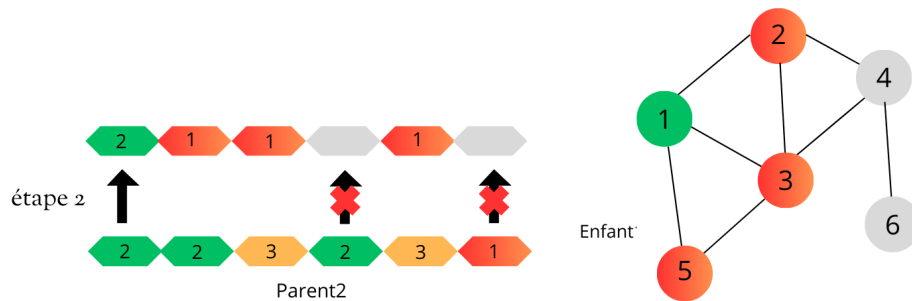


Figure 10: *Crossover étape 2*

Celui-ci est utilisé pour remplir les gènes restants dans le génome enfant. Cependant, cette opération mène à une intersection entre l'attribution de communauté du parent 1 et du parent 2 et c'est à ce moment que la validité du génome enfant n'est plus garantie. L'exemple de la *Figure 10* met en lumière les deux cas possibles où le transfert d'un gène du parent 2 est impossible vers

l'enfant 1. En effet, la communauté 2 dans le parent 2 représentait une communauté valide, car elle était composée des nœuds 1,2,4 qui sont voisins entre eux. Cependant suite à l'étape 1 le nœud numéro 2, qui fait la liaison entre le 1 et le 4 a déjà été attribué à la communauté 1, il est donc impossible ici d'attacher à la fois 1 et 4 dans la communauté 2. L'autre cas de conflit possible est lorsque le gène à transmettre est un gène représentant la même communauté que celle choisi avec le parent 1. Celle-ci était forcément connectée dans le parent 1, mais lui ajouter un nœud qui n'est voisin avec aucun d'entre eux créer une non-viabilité.

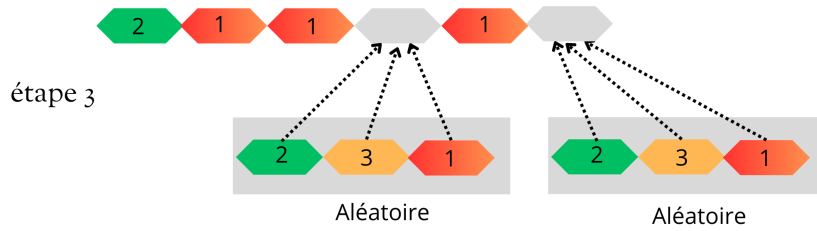


Figure 11: *Crossover étape 3*

La solution que nous avons adoptée est de remplir, dans l'étape 3, les gènes restant par une communauté aléatoire jusqu'à en trouver une qui satisfasse le génome, de la même manière que lorsqu'on initialise la population initiale. Comme pour la sélection, on introduit un taux de croisement qui déterminera quel pourcentage de la population va se reproduire. Le taux le plus fréquemment utilisé est de 0.80, nous avons aussi choisi d'utiliser ce taux, car il donnait des résultats satisfaisants.

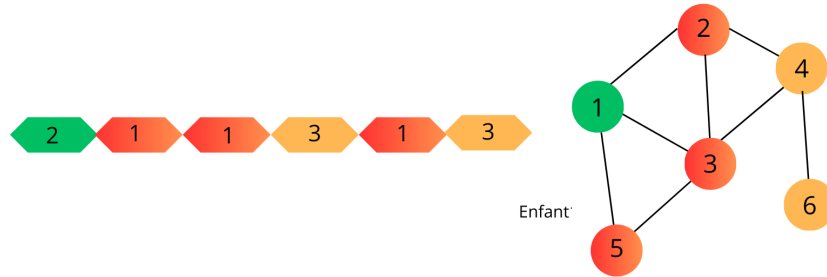


Figure 12: *Crossover, solution finale possible*

2.4.3 Mutation

La mutation est un opérateur qui permet à certains génomes d'évaluer de nouvelles régions dans l'espace des solutions pour potentiellement s'améliorer. Nous avons fixé le taux de mutation à 0.05. Dans notre contexte, la mutation va

être de choisir un gène aléatoirement dans le génome à muter, puis de changer l'attribution de communauté de ce nœud. Au lieu de choisir, n'importe quelle communauté nous avons décidé de regarder la couleur des voisins du nœud à muter. En effet le but ici de maximiser le nombre de nœuds de même couleur au sein d'une même communauté. Il y a donc 3 cas possibles :

- Soit le nœud choisi n'a aucun voisin de même couleur que lui : dans ce cas, il rejoint une communauté au hasard parmi celles attribuées à ses voisins.
- Soit il n'a qu'un voisin de même couleur : il rejoint la communauté de celui-ci.
- Soit il a plusieurs voisins de même couleur : il rejoint une de leurs communautés aléatoirement.

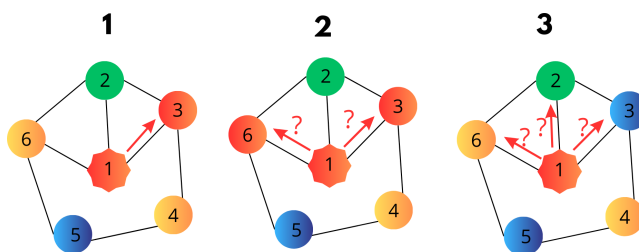


Figure 13: *Mutation*

Il est possible que tous les voisins du nœud mutant aient la même communauté que lui et selon notre méthode, il rejoindra une de leur communauté, c'est-à-dire que rien n'aura changé par rapport à avant la mutation. Ce cas est difficile à aborder, car un nœud mutant qui est en plein milieu d'une communauté ne peut pas changer la sienne sans prendre le risque de créer une déconnexion et de rendre le génome non-valide. Ce problème peut s'éviter en choisissant quel nœud doit muter, mais cela enlèverait alors la seule opération stochastique de la mutation qui est la sélection d'un gène au hasard. En réalité, ce problème est négligeable, car un nœud qui change de communauté au milieu d'une autre a une chance presque nulle d'améliorer la solution.

3 Résultats

Chacune des exécutions effectuées pour ces tests ont eu les paramètres suivants:

- Taille de la population: 5 000
- Taux Darwinien: 1

- Taux de croisement: 0.8
- Taux de mutation: 0.05

Le nombre de générations varie selon les besoins de chaque test.

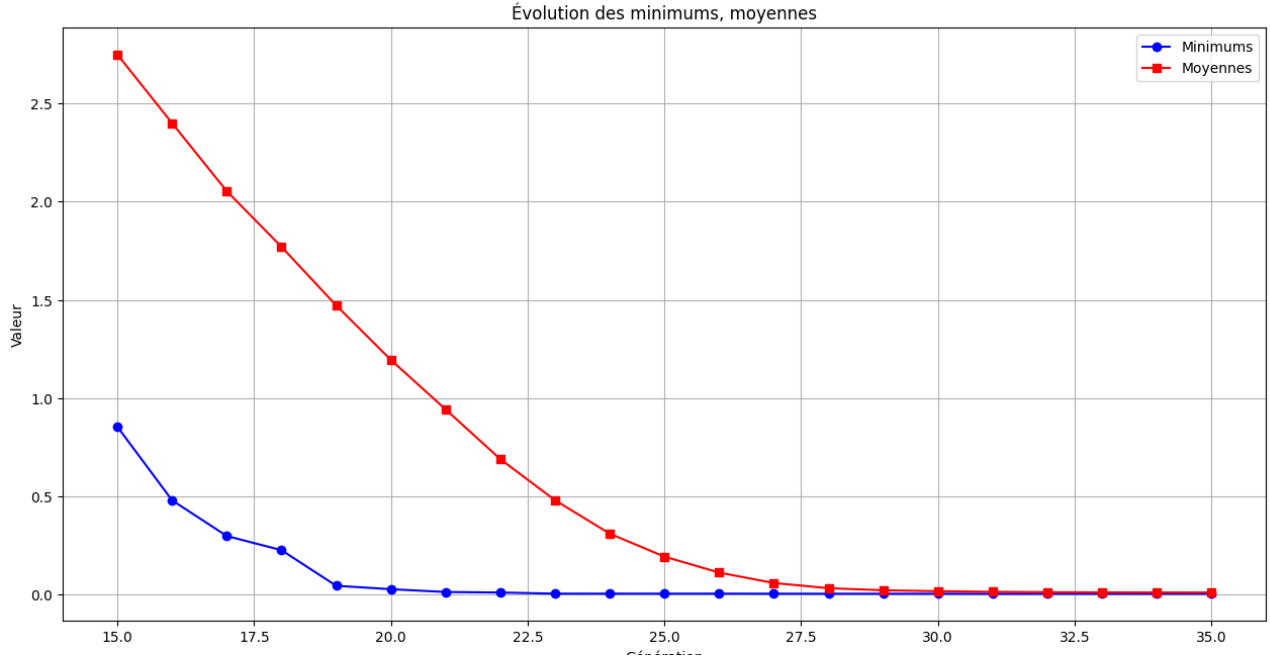


Figure 14: *Evolution de l'entropie minimum et moyenne par génération*

Ce graphique correspond à la moyenne des entropies minimales et moyennes par génération et sur 10 graphes *Small World* de taille 50 différents. Les générations de 0 à 15 ne contiennent que des génomes avec des scores d'entropie infinie. On retient à chaque génération le meilleur génome pour le mettre dans la génération suivante, c'est pourquoi la courbe des minimums ne fait que baisser. On remarque que plusieurs génomes avec une entropie non-infinie apparaissent en même temps, car le minimum est éloigné de la moyenne à la génération 15. La moyenne baisse au fil des itérations ce qui suggère une amélioration globale des générations. De plus, l'entropie minimale ne fait que diminuer jusqu'à ce que la moyenne soit exactement égale à elle ; dans ce cas, le meilleur génome ne changera jamais ce qui veut dire qu'on a atteint un minimum local.

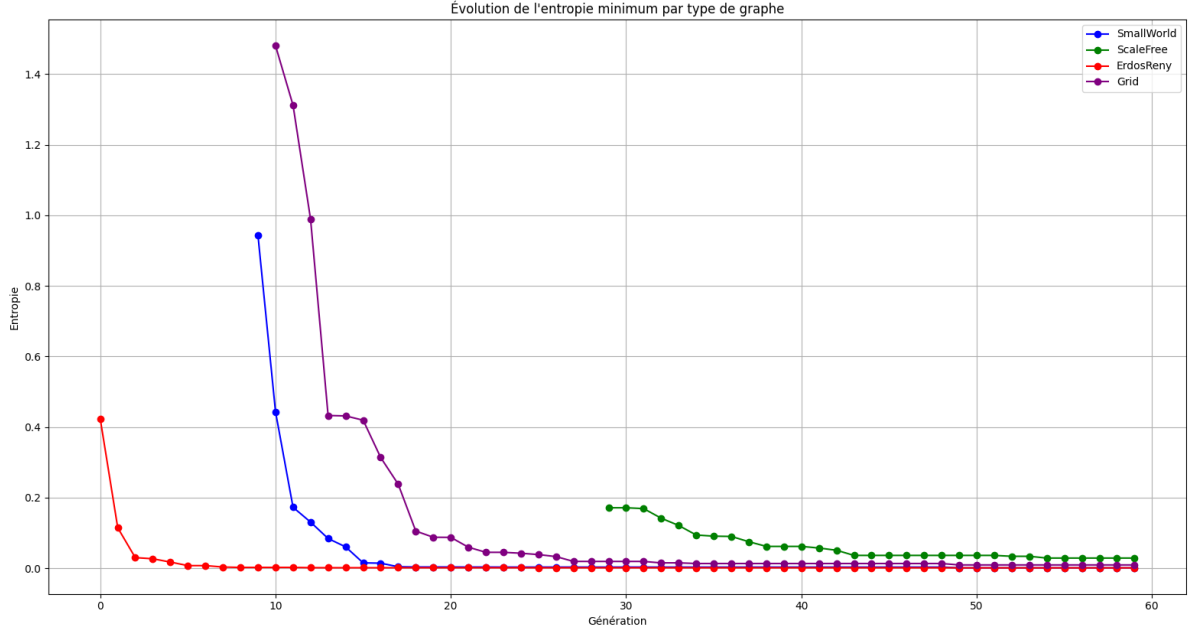


Figure 15: *comparaison sur plusieurs type de graphes*

Ce graphique représente l'évolution de la moyenne, sur 10 essais, des entropies minimums par générations sur différents types de graphes. Ces exécutions ont été effectuées avec une taille de population de seulement 1 000 à cause de la grande quantité de calcul nécessaire, ce qui explique les résultats aux alentours des $10e-2$. On remarque que la topologie du graphe en entrée va changer notamment la première génération où l'entropie ne sera pas infinie, la vitesse de convergence ainsi que l'entropie finale trouvée.

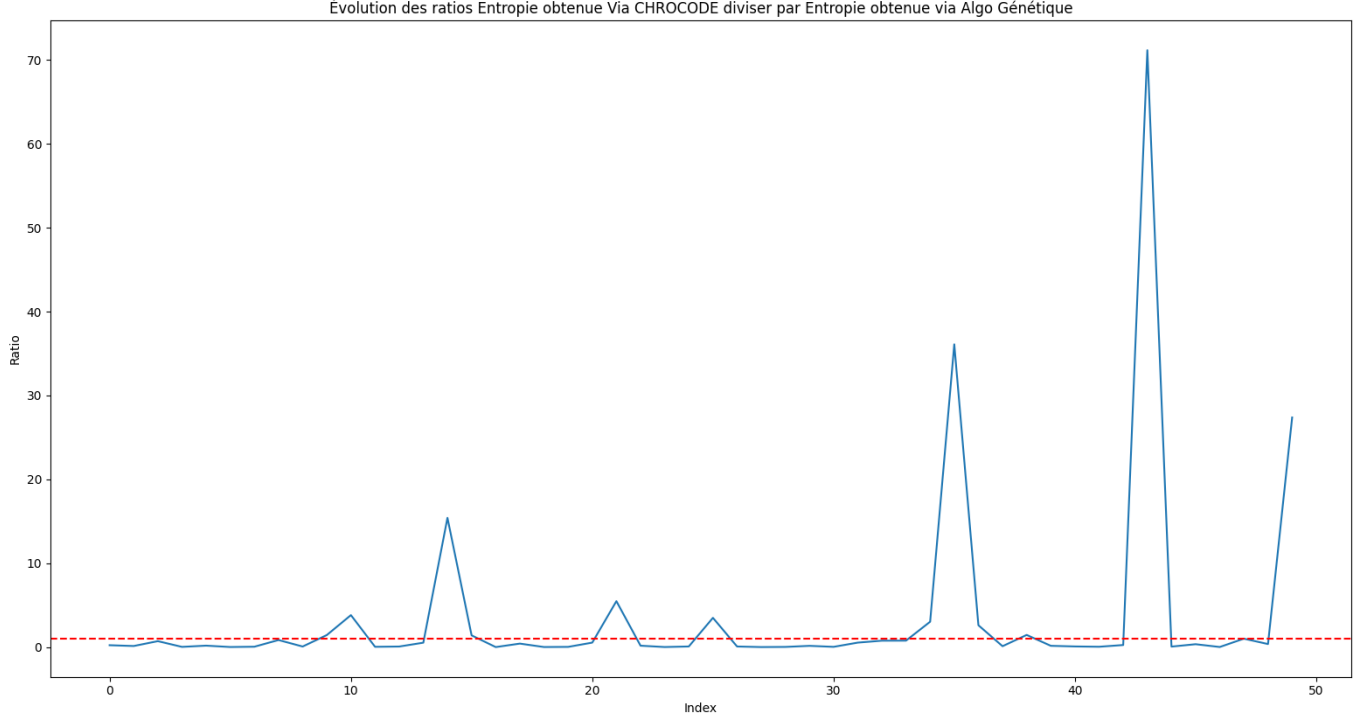


Figure 16: *Entropie CHROCODE diviser par Entropie Algorithme génétique*

Afin de comparer les deux approches AG et Chrocode, nous générons un graphique qui compare les deux méthodes de partitionnement des nœuds colorés dans les graphes, en utilisant la formule suivante :

$$\text{Ratio } R = \frac{\text{Entropie Chrocode (EC)}}{\text{Entropie AG (EA)}} \quad (5)$$

Nous cherchons à minimiser l'entropie donc:

Si $R > 1$, AG est considéré comme meilleur, tandis que

Si $R < 1$, CHROCODE est jugé supérieur.

Les 50 itérations ont tous été faites sur des profils de couleurs différents et des types de graphes différents. Ce graphique nous permet de conclure que les 2 méthodes sont équivalentes quant au nombre de fois où l'une est supérieure à l'autre.

L'avantage de l'algorithme génétique est sa flexibilité. En effet, la modification des opérateurs peut significativement optimiser les résultats. Pour essayer de l'améliorer et d'avoir un algorithme qui est capable de surpasser l'algorithme Chrocode, mais également les résultats déjà obtenus, nous avons

changé l'opérateur de croisement. Au lieu de compter le nombre de nœuds dans chacune des communautés, nous décidons plutôt de calculer l'entropie de chacune des communautés du génome est de donner l'avantage à celle qui a la plus petite.

Cela n'a pas changé drastiquement les résultats et la performance globale de l'algorithme reste la même.

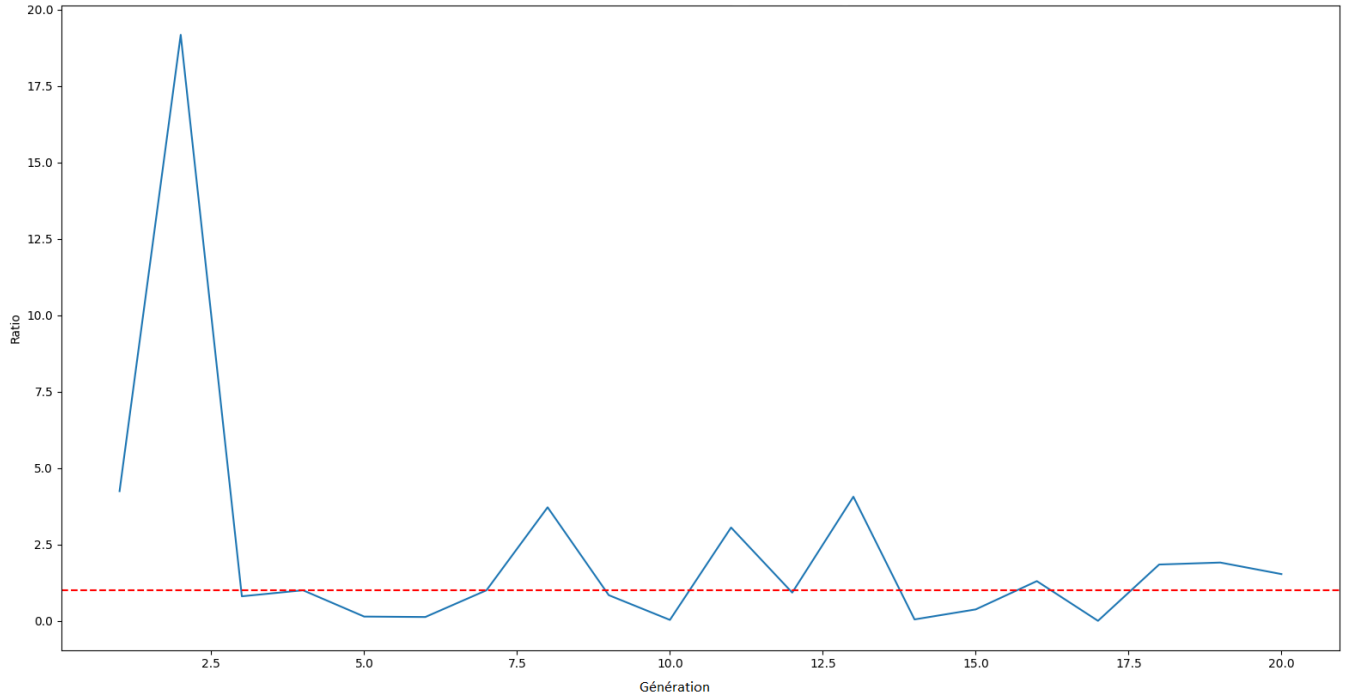


Figure 17: *Selection par entropie divisier par Selection par nombre de noeuds*

4 Conclusion

Pour conclure, l'adoption de l'approche métaheuristique des algorithmes génétiques pour optimiser l'algorithme CHROCODE et détecter des communautés colorées dans des graphes s'est avérée prometteuse. Nous avons pu obtenir des résultats comparables à ceux de l'algorithme Chrocode. De plus, l'un des avantages majeurs de cette approche par rapport à l'algorithme CHROCODE réside dans sa capacité à pouvoir être raffiné, au prix d'un temps de calcul beaucoup plus long.

Une amélioration évidente est de changer le critère d'arrêt de l'algorithme

qui est actuellement un simple nombre d'itération. Nous avons pensé à le remplacer par un nombre d'itération à partir duquel la meilleure entropie ne change pas. Pour permettre à cette approche d'être efficace, il faut un algorithme qui puisse sortir des minimums locaux. Comme on l'a vu lors des résultats, lorsque la moyenne des performances d'une génération est la même que sa meilleure valeur, l'algorithme ne s'améliorera plus. L'un des axes majeurs d'évolution de notre travail est donc de résoudre ce problème. Cela peut être fait par la modification des opérateurs dans le but d'offrir plus de diversité génétique.

Vous retrouverez le code à cette adresse: <https://github.com/suprawall/AGCodeTER>

Bibliographies

- [1] Franck Delaplace. “Entropic Detection of Chromatic Community Structures”. In: (2023), pp. 1–28.
- [2] Franck Delaplace. “Genetic Algorithm”. In: (2023), pp. 1–52.
- [3] Franck Delaplace. “Meta Heuristic”. In: (2023), pp. 1–29.

.